

$$P(x) = \sum_{i=0}^n y_i \cdot \ell_i(x), \quad \ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

$$X_k = \sum_{j=0}^{n-1} a_j \cdot \omega^{jk} \pmod{p}$$

ω is an n -th root of unity in F_p

Security Review Report

NM-1019 - PXE-Bridge

Axol.io

September 4, 2026



NETHERMIND
SECURITY

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	Aztec Background	4
4.1.1	Every Account is a Contract	4
4.1.2	Two Execution Phases, Two Kinds of State	4
4.1.3	AppPayload — the “What to Execute” Bundle	4
4.1.4	Authwit — the Delegated-Authorization Side Door	5
4.2	The SpendingLimitAccount Contract	5
4.2.1	Storage and Configuration	5
4.2.2	Deployment and Initialisation	6
4.2.3	Entrypoint (Private)	6
4.2.4	The Declared-vs-Actual Binding	7
4.2.5	check_spending_public (Public)	7
4.2.6	Administration and Timelock	7
4.2.7	Authwit Disabled	8
4.2.8	One File, One Contract, Three Compile Targets	8
4.2.9	Diff Against the Stock SchnorrAccount	8
4.2.10	The TypeScript Counterpart	9
4.2.11	Build, Artifact, and Test Coverage	9
5	Risk Rating Methodology	10
5.1	ZK Audit Issue Classification	10
6	Issues	11
6.1	[High] A single set of limits cannot govern different tokens	11
6.2	[High] Daily counter can be reset to zero	11
6.3	[High] u64 limits are too small for 18-decimal tokens	12
6.4	[Medium] Fixed Epoch Daily Limit Allows Up to 2× Cap Bypass	13
6.5	[Medium] Recipient is published in public calldata	14
6.6	[Low] cancellable Transactions Cannot Be Cancelled	14
6.7	[Low] Documented timelock does not cover the allowlist	15
6.8	[Info] The single-call restriction in entrypoint may interfere with deployment	15
7	Documentation Evaluation	17
8	Test Suite Evaluation	18
8.1	Tests Output	18
9	About Nethermind	19

1 Executive Summary

This document presents the security review performed by [Nethermind Security](#) for the **SpendingLimitAccount Noir contract** implemented in [pxe-bridge](#) by **Xochi**. Xochi is a private execution venue on Ethereum: users sign intents, statements of a desired outcome rather than a transaction, and competing off-chain solvers fill them, keeping the order out of the public mempool, with regulatory posture carried by zero-knowledge compliance proofs rather than by disclosure. [pxe-bridge](#) is the Aztec settlement leg of that architecture: a JSON-RPC server that embeds an Aztec PXE wallet so that EVM-native solvers, which have no concept of local proving, encrypted notes, or Grumpkin keys, can deliver shielded transfers on Aztec L2 through an interface they already speak. Because the solver fills each intent immediately with its own capital, the account behind that server is a hot wallet pre-funded with days of expected settlement volume, and its signing key is the whole of the system's custody risk.

SpendingLimitAccount exists to make the compromise of that key survivable. Aztec has native account abstraction, every account is a contract, and what constitutes a valid transaction is programmable per account, so the spending policy that would otherwise live in the server process, alongside the key it is meant to constrain, is instead welded into the account itself, where the network enforces it. The contract is a stock Aztec SchnorrAccount extended with a per-transaction amount ceiling, an epoch-based daily volume cap, a fail-closed recipient allowlist, and a timelocked administrative path for changing those parameters. Its structure is dictated by the protocol: the limits are mutable public state and therefore cannot be read by the private endpoint, so the endpoint enqueues a public call that checks them and relies on atomic revert to make an after-the-fact check as binding as a prior one. The delicate part, and the contract's most substantive original engineering, is the *declared-versus-actual binding* — an in-circuit proof that the amount and recipient handed to the public check are exactly the ones the executed transfer carries, since the party supplying them is precisely the party the threat model assumes is compromised.

The audited code comprises 365 lines of code written in Noir. The audit was performed using (a) manual analysis of the codebase and (b) automated analysis tools.

Along this document, we report eight issues: three as High, two as Medium, two as Low, and one as Informational. **Fig. 1** summarizes the issues identified in *the smart contract*.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 concludes the document. Appendix describes the review of the deployment scripts.

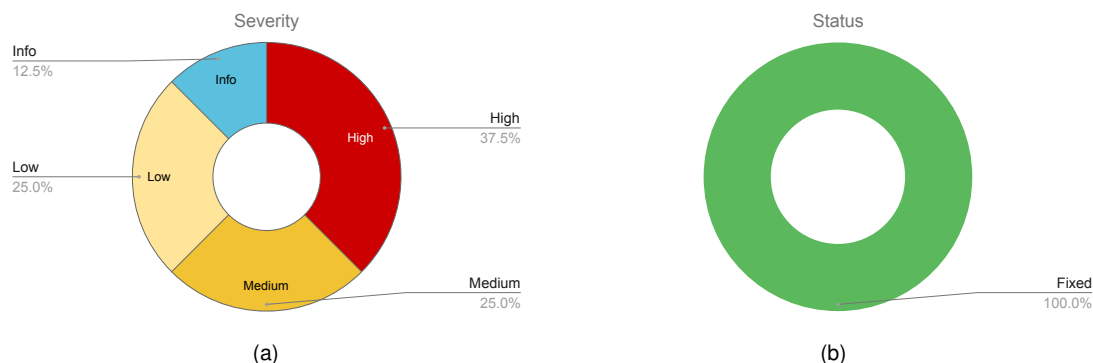


Fig. 1: Distribution of issues: Critical (0), High (3), Medium (2), Low (2), Undetermined (0), Informational (1), Best Practices (0). Distribution of status: Fixed (8), Acknowledged (0), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	August 13, 2026
Final Report	September 4, 2026
Response from Client	Regular responses during audit engagement
Repository	pxe-bridge
Audit Commit	583986887337bb34cedf34f938c8f9bf03b87379
Final Commit	f0f8d82875683a27178eab235326d9423218cb4f
Documentation	Low
Documentation Assessment	Medium
Test Assessment	Low

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	contracts/spending_limit_account/main.nr	358	123	34.3%	64	545
2	contracts/spending_limit_account/public_key_note.nr	7	2	28.6%	1	10
	Total	365	125	34.2%	65	555

3 Summary of Issues

	Finding	Severity	Update
1	A single set of limits cannot govern different tokens	High	Fixed
2	Daily counter can be reset to zero	High	Fixed
3	u64 limits are too small for 18-decimal tokens	High	Fixed
4	Fixed Epoch Daily Limit Allows Up to 2× Cap Bypass	Medium	Fixed
5	Recipient is published in public calldata	Medium	Fixed
6	cancellable Transactions Cannot Be Cancelled	Low	Fixed
7	Documented timelock does not cover the allowlist	Low	Fixed
8	The single-call restriction in entrypoint may interfere with deployment	Info	Fixed

4 System Overview

Xochi is a private execution venue on Ethereum: users sign *intents*, which are statements of a desired outcome, and competing off-chain **solvers** fill them, keeping the order out of the public mempool. Identity at the front door is pseudonymous (**XID**, a WebAuthn passkey whose session token is SHA-256(credentialId), with no username, email, or wallet address stored), and regulatory posture is meant to be carried by zero-knowledge compliance proofs (**ERC-8262**) generated on the user's device rather than by disclosure. Settlement is tiered: L1 stealth addresses (ERC-5564) at the base, and, for the Private and Sovereign tiers, shielded settlement on Aztec L2.

pxe-bridge is the Aztec leg of that architecture, and the subject of this engagement. A solver's stack is EVM-native and has no concept of local proving, encrypted notes, or Grumpkin keys. The bridge embeds a PXE wallet and exposes it behind a single JSON-RPC method, `aztec_createNote`, which performs exactly one on-chain operation: `transfer_to_private(to, amount)` — moving tokens from the bridge account's public Aztec balance into a private note held by the recipient. Both endpoints are on L2.

The security problem follows directly from that role. To fill an intent's destination leg immediately, the solver advances its own capital. The bridge's Aztec account is therefore a hot wallet pre-funded with days of expected settlement volume, and the server holding its signing key is in potential risk of being compromised.

SpendingLimitAccount is the answer to that gap. Aztec has native account abstraction. Every account is a contract, and what counts as a valid transaction is programmable per account, so the same policy can be moved into the account itself, where the network, not the compromised process, enforces it. The **SpendingLimitAccount** introduces the spending limit and the restricts calldata to match the whitelisted recipients, to limit the potential damages made by the attacker who hold the compromised key.

4.1 Aztec Background

4.1.1 Every Account is a Contract

Aztec has native account abstraction: there are no externally owned accounts, and a transaction carries no protocol-level signature. Instead, the transaction's first call is a call to the sender's own account contract — its **entrypoint** — and that contract decides, in its own circuit, whether the transaction is authorized. Because validation runs client-side as a ZK proof, Aztec can afford unlimited validation complexity without affecting network performance: nobody else ever executes your entrypoint.

Two consequences shape everything below. First, authorization policy is *programmable*, which is what makes an on-chain spending cap possible at all. Second, replay protection is the *account's* job — Aztec does not enforce sequential nonces.

4.1.2 Two Execution Phases, Two Kinds of State

A transaction splits into a **private phase**, executed locally in the PXE (Private eXecution Environment) and proved, and a **public phase**, executed by the network afterwards in the AVM. State variables are typed to match: `PublicMutable<T>` can be read and written *only* in public functions; `PublicImmutable<T>` is write-once and, because it cannot have changed, is also readable privately; `SinglePrivateImmutable<Note>` holds one immutable private note; `Map<K, V>` derives a slot per key, so an unwritten key reads as V's zero value.

This typing dictates the contract's entire architecture. Spending limits are mutable and must be visible to the sequencer, so they live in `PublicMutable` — and therefore **cannot be checked inside the private entrypoint**. The entrypoint can only *enqueue* a public call that does the checking. What makes that as strong as a before-the-fact check is atomicity: if any enqueued public call asserts, the whole transaction reverts, including every private effect.

4.1.3 AppPayload — the “What to Execute” Bundle

The entrypoint receives an `AppPayload`: a fixed array of `ACCOUNT_MAX_CALLS = 5` function calls plus a `tx_nonce`.

```
struct FunctionCall {
    args_hash: Field,           // offset 0
    function_selector: FunctionSelector, // offset 1
    target_address: AztecAddress, // offset 2
    is_public: bool,           // offset 3
    hide_msg_sender: bool,     // offset 4
    is_static: bool,           // offset 5
}
```

`execute_calls()` iterates the five slots and **skips any call whose target_address is zero** — that is how a payload with fewer than five calls is encoded. `args_hash` for a private call is `hash_args(args)`, i.e. `poseidon2_hash_with_separator(args, DOM_SEPARATOR_FUNCTION_ARGS)`, or 0 for an empty argument list. These facts — five slots, six fields each, zero target means empty, `args_hash` at offset 0 — are the foundation of the declared-vs-actual binding described below.

4.1.4 Authwit — the Delegated-Authorization Side Door

An **authentication witness** lets a third-party contract act on an account's behalf for one specific action. The scheme is two-level: an *inner hash* $H(\text{caller}, \text{selector}, \text{args_hash})$ names who is doing what with which arguments, and a *message hash* $H(\text{consumer}, \text{chain_id}, \text{version}, \text{inner_hash})$ binds it to a chain, a protocol version, and the consuming contract. When a token contract needs authorization it makes a *static* call to the account's `verify_private_authwit(inner_hash)` and expects the magic value `IS_VALID_SELECTOR`.

This matters here because authwit is a **second, independent path to move the account's funds that does not route through entrypoint**. Any spending policy implemented only in the entrypoint is bypassable through it.

4.2 The SpendingLimitAccount Contract

The contract is a stock Aztec SchnorrAccount with a spending policy welded into its entrypoint. It is not a guard or proxy sitting in front of a wallet — it *is* the wallet: it holds the funds, verifies the Schnorr signature, and originates every transaction. That is why the policy cannot simply be "called around." Two entry points, in two execution environments, within a single transaction:

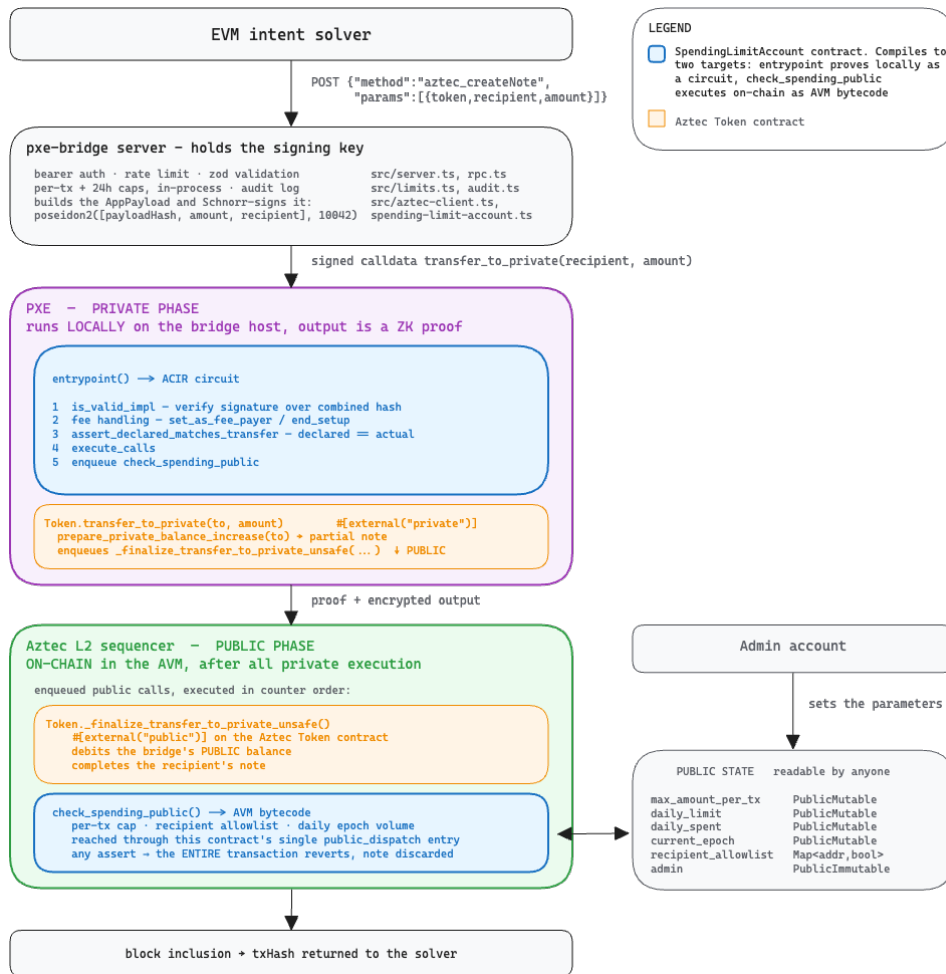


Fig. 1: Architecture of PXE bridge.

Out of band, on their own transactions from a separate admin account, sit four policy-management functions — `set_recipient`, `propose_limits`, `apply_limits`, `reset_daily_spent`. These never run inside a transfer.

4.2.1 Storage and Configuration

```

signing_public_key : SinglePrivateImmutable<PublicKeyNote> // private
max_amount_per_tx  : PublicMutable<Field>
daily_limit        : PublicMutable<Field>
daily_spent        : PublicMutable<Field>
current_epoch      : PublicMutable<Field>
recipient_allowlist : Map<AztecAddress, PublicMutable<bool>>
admin              : PublicImmutable<AztecAddress> // write-once
pending_max_amount : PublicMutable<Field>
  
```

```

pending_daily_limit : PublicMutable<Field>
pending_change_block : PublicMutable<Field>
initialized          : PublicMutable<bool>

BLOCKS_PER_EPOCH:    u64 = 7200;    // ~24h at 12s/block
TIMELOCK_BLOCKS:     u64 = 5;       // ~60s at 12s/block
DOM_SEP__SPENDING_LIMIT: u32 = 10042;

```

Only `signing_public_key` is private — a Grumpkin point stored as a note, identical to the stock account's. Everything else — the caps, the daily usage, the allowlist membership, the admin — is **public on-chain state, readable by anyone**. That is a deliberate trade: the policy becomes auditable by third parties at the cost of publishing the bridge's risk parameters and its aggregate daily throughput. admin being `PublicImmutable` means it **cannot be rotated**.

Note the semantics of **zero as “disabled”**: `max_amount_per_tx == 0` and `daily_limit == 0` each mean *no such limit*, not a *limit of zero*. `DOM_SEP__SPENDING_LIMIT = 10042` is a locally invented constant, duplicated by hand on the TypeScript side; Aztec's own separators are hash-derived 32-bit values in the 10^8 – 10^9 range, so it does not collide in v4.2.0, but it follows none of the protocol's conventions.

4.2.2 Deployment and Initialisation

The `private` constructor stores the public-key note and, because it cannot itself write `PublicMutable` state, *enqueues* `initialize_public_state(max, daily, admin)`. If that public call asserts, the whole deployment reverts, so there is no window in which the account exists with uninitialized limits. `initialize_public_state` is guarded by `msg_sender == this_address` and `!initialized`, range-checks each non-zero limit through a `u64` round-trip, and requires `daily_limit >= max_per_tx` when both are non-zero.

The allowlist starts **empty** and is **always enforced**, so the deployment order in the contract's header comment is not optional:

- | | |
|--|---|
| 1. Deploy | constructor stores key, enqueues public init |
| 2. <code>set_recipient(r, true)</code> | for every recipient, sent by the ADMIN account |
| 3. Transact | any transfer to a non-allowlisted recipient reverts |

Two operational consequences follow. The admin must be a *separate, funded Aztec* account — the bridge cannot allowlist recipients for itself, because routing `set_recipient` through its own entrypoint would require the enqueued `check_spending_public` to pass, which requires the target to be allowlisted already. And losing the admin key freezes policy permanently: the allowlist can never change, limits can never be re-proposed, and `reset_daily_spent` becomes unavailable.

4.2.3 Entrypoint (Private)

The signature diverges from stock by two parameters, and the body — unlike the stock contract, which delegates to `AccountActions::entrypoint` — is reimplemented inline so those parameters can enter the signed hash:

```

fn entrypoint(
  app_payload: AppPayload,
  fee_payment_method: u8,
  cancellable: bool,
  declared_amount: Field,           // <- added
  declared_recipient: AztecAddress, // <- added
)

```

1. Signature verification over a combined hash. Stock Schnorr signs $H(\text{this}, \text{chain_id}, \text{version}, \text{payload_hash})$. Here the signed preimage additionally covers the declared pair:

```

let combined_hash = poseidon2_hash_with_separator(
  [payload_hash, declared_amount, declared_recipient.to_field()],
  DOM_SEP__SPENDING_LIMIT);
let message_hash = compute_authwit_message_hash(
  this_address, chain_id, version, combined_hash);
assert(is_valid_impl(self.context, message_hash), "Invalid signature");

```

A captured signature is therefore bound to one (payload, amount, recipient) triple, and the message is domain-separated from a stock Schnorr message. What this does *not* give is binding to reality: the key holder chooses the declared values *and* signs them. That gap is closed in step 3.

2. Fee handling, verbatim from upstream: `PREEXISTING_FEE_JUICE → set_as_fee_payer() + end_setup(); FEE_JUICE_WITH_CLAIM → set_as_fee_payer()`. Fees are paid in Fee Juice, a separate asset, and are not counted against the spending limits.

3. `assert_declared_matches_transfer(...)` — the new security-critical step, below. **4.** `execute_calls` runs the payload. **5.** `check_spending_public` is enqueued. **6.** a cancellation nullifier is pushed if cancellable.

`#[noinitcheck]` is stock — the entrypoint must be callable in the same transaction that deploys the account — and the initialization guarantee is restored downstream by `check_spending_public`'s `assert(initialized)`.

4.2.4 The Declared-vs-Actual Binding

This is the contract's single most delicate property. The public check runs in a separate context and cannot see the private call it is policing, so it is handed `declared_amount` and `declared_recipient` and enforces the limits against *those* — values supplied by exactly the party the threat model assumes is compromised. Without more, an attacker signs `declared_amount = 1` alongside a payload that transfers the whole balance, every check passes, and the caps are decorative.

```
fn assert_declared_matches_transfer(payload, declared_amount, declared_recipient) {
  let expected = hash_args([declared_recipient.to_field(), declared_amount]);
  let serialized: [Field; 31] = payload.serialize(); // 5 * 6 + tx_nonce
  assert_single_call_matches(serialized, expected);
}

fn assert_single_call_matches(serialized: [Field; 31], expected_args_hash: Field) {
  let mut nonempty = 0; let mut matching = 0;
  for i in 0..5 {
    let base = i * 6;
    if serialized[base + 2] != 0 { // target_address
      nonempty += 1;
      if serialized[base] == expected_args_hash { matching += 1; }
    }
  }
  assert(nonempty == 1, "Expected exactly one call in payload");
  assert(matching == 1, "Transfer does not match declared spending");
}
```

The reasoning chain is: $31 = 5 \times 6 + 1$; `target_address == 0` is exactly the predicate `execute_calls` uses to skip a slot, so “non-empty” here and “executed” there are the same set; requiring `nonempty = 1` means precisely one call runs, so no hidden second transfer can ride along; and `transfer_to_private(to: AztecAddress, amount: u128)` takes exactly two arguments, each encoding to one field, so equality of `args_hash` pins both values by Poseidon2 preimage resistance. The single executed call therefore receives exactly `[declared_recipient, declared_amount]`.

What the guard reads is **offset 0** (`args_hash`) and **offset 2** ($\neq 0$). It does not read `function_selector` (offset 1), the *value* of `target_address`, or the three booleans. So the property established is “the one executed call takes arguments `[allowlisted_recipient, declared_amount]`” — not “the one executed call is `transfer_to_private` on the intended token.” The callee is unconstrained, which is the root of several findings.

4.2.5 check_spending_public (Public)

Guarded by `msg_sender == this_address` and `assert(initialized)`, then three checks in order:

- Per-transaction cap** — skipped entirely when `max_per_tx == 0`; otherwise `assert_amount_within_limit(declared_amount, max_per_tx)`, which round-trips both operands through `u64` before comparing.
- Recipient allowlist** — *always* enforced. The zero address is rejected outright so it cannot serve as a “skip the allowlist” sentinel (an earlier revision used it that way), and `Map` returns `false` for unwritten keys. A freshly deployed account can therefore transfer to nobody until the admin acts. This is fail-closed, and it is the single most important operational fact about the contract.
- Daily volume** — skipped when `daily_limit == 0`. The window is *fixed*, keyed on `block_number / BLOCKS_PER_EPOCH`: when the epoch differs from the stored one the counter resets to zero, otherwise the declared amount is added to `daily_spent` and the *sum* is checked against the limit.

Note the contrast with `src/limits.ts`, which uses a true 24h sliding window, and note that in (c) the range check applies to the accumulated sum rather than to the addend — the addend is range-checked only on the (a) path.

Remark about the two limit layers

The repository presents `src/limits.ts` and this contract as defence-in-depth: an application-level layer that bounds bugs and rogue callers, and an on-chain layer that survives key compromise. The framing is right, but the two are not equivalent controls, and on several axes the on-chain layer is the *weaker* of the pair. The application layer uses a true sliding 24h window and bigint arithmetic; the contract uses a fixed epoch on absolute block height and `u64` range checks. Neither layer has a token-denomination concept, so a cap tuned for a 6-decimal stablecoin is roughly 10^{12} times too permissive against an 18-decimal asset — in *both* layers simultaneously, which is precisely the correlation defence-in-depth is supposed to avoid. The on-chain layer's real, irreplaceable contribution is the recipient allowlist, which has no application-level counterpart at all.

4.2.6 Administration and Timelock

Function	Admin check?	Timelocked?
<code>propose_limits(new_max, new_daily)</code>	yes	sets <code>block + 5</code>
<code>apply_limits()</code>	no	requires <code>block >= pending</code>
<code>set_recipient(recipient, allowed)</code>	yes	no -- immediate

reset_daily_spent()	yes	no
---------------------	-----	----

apply_limits being permissionless is a normal timelock pattern — the values were set by the admin, anyone may push the button once the delay elapses — and it re-validates `new_daily >= new_max` before clearing the pending slots. `set_recipient` takes effect immediately. `reset_daily_spent` exists because the epoch reset is block-height-driven and stalls if block production stalls; it also means the daily cap is only as strong as the admin key, which can zero the counter at will and without delay.

4.2.7 Authwit Disabled

```
#[external("private")] #[noinitcheck] #[view]
fn verify_private_authwit(_inner_hash: Field) -> Field { 0 }
```

Stock returns `IS_VALID_SELECTOR`; this returns `0`, which no consumer accepts. This is the correct and necessary decision — a valid authwit would let a token contract move the account's funds without routing through `entrypoint`, bypassing every spending check — and the bridge's own transfers execute through `entrypoint` with `msg_sender == this` account, so nothing is lost for the intended use case. It is, however, the one place the contract *removes* native functionality rather than adding to it, and the consequences reach further than the `entrypoint`: the two fee-payment methods that require an authwit can never work with this account. `lookup_validity`, the unconstrained off-chain query helper, was left as a verbatim copy of the stock implementation and still reports true for a valid, unspent signature.

4.2.8 One File, One Contract, Three Compile Targets

"The same code runs locally and on L2" is not what happens. `main.nr` is a single contract with a single on-chain registration, but each function compiles to a different execution target chosen by its attribute:

Attribute	Compiles to	Executes
-----	-----	-----
<code>#[external("private")]</code>	ACIR (constrained)	locally in the PXE; output=proof
<code>#[external("public")]</code>	Brillig -> AVM	ON-CHAIN, by the sequencer
<code>#[external("utility")]</code>	Brillig	locally only, never proved
<code>#[contract_library_method]</code>	inlined	wherever the caller runs
<code>#[test]</code>	test harness	nargo test

The last two rows do not appear in the compiled artifact at all: `assert_declared_matches_transfer` is not a callable function, it is literally part of `entrypoint`'s circuit, which is why the binding cannot be skipped. And "deployed" means less than it usually does. **Private functions are never deployed** — the chain stores only a selector and a verification-key hash per private function, and your host holds the bytecode. **Public functions are deployed as one entry**: the on-chain `packedBytecode` is the `public_dispatch` function, and all six public functions are reached through it by selector. **Utility functions are neither**; they are committed only inside `artifactHash`.

The asymmetry is the load-bearing fact of the review: an attacker who owns the server runs the PXE and produces the proof, so they control the private half entirely. They cannot touch the public half. **That is the entire security value of this contract.**

4.2.9 Diff Against the Stock SchnorrAccount

Upstream reference: [@ v4.2.0](https://github.com/noir-projects/noir-contracts/contracts/account/schnorr_account_contract).

Element	Stock	This contract
-----	-----	-----
Storage	signing_public_key only	+ 10 public slots
constructor	2 args	5 args; enqueues public init
entrypoint	<code>AccountActions::entry..</code>	inlined; + declared amount/recipient
Signed preimage	<code>H(this,cid,ver,payload)</code>	<code>H(this,cid,ver,poseidon2([payload, amount,recipient],10042))</code>
<code>verify_private_..</code>	<code>IS_VALID_SELECTOR</code>	<code>0 -- DISABLED</code>
<code>is_valid_impl</code>	Schnorr verify	identical
<code>lookup_validity</code>	as-is	identical (now inconsistent)
Public functions	none	6 (init, check, propose, apply, set_recipient, reset_daily)

Everything the stock contract does, this contract still does; the additions are purely restrictive except for the six public functions, which are the new attack surface and are all guarded (`msg_sender == this_address` for the two internal ones, `msg_sender == admin` for three, timelock for `apply_limits`).

For calibrating scrutiny, the 409 non-blank, non-comment lines under `contracts/` divide into three tiers: **58 lines (14%)** of framework imports and scaffolding, whose correctness is Aztec's problem; **95 lines (23%)** copied verbatim from the stock account (`lookup_validity`, `is_valid_impl`, the note-storage tag dance, fee handling, the cancellation nullifier, `public_key_note.nr`) — low risk in itself, but now a *fork* that will not track upstream fixes; and **256 lines (63%)** that are genuinely custom, where every finding in this report lives. Not one finding is a defect in the framework tier.

4.2.10 The TypeScript Counterpart

`src/spending-limit-account.ts` is the client-side half — it builds and signs the arguments that `main.nr` checks — and the two must agree simultaneously on five hand-maintained invariants:

#	Invariant	Enforced by
1	DOM_SEP 10042	nothing -- two hand-kept literals
2	Combined-hash preimage/order	nothing -- and duplicated across two call sites
3	Entrypoint ABI	nothing -- ~90 lines of hand-written FunctionAbi
4	Constructor args and order	nothing
5	FunctionCall serialize layout	nargo test function_call_serialize_layout

Only invariant 5 has a test — and it is a good one, converting a silent “framework changed its struct order and the guard now reads garbage” failure into a red CI job. Invariants 1–4 are duplicated constants and a duplicated ABI that will drift on the next SDK bump; their failure modes are loud (“Invalid signature” on every transaction, or a selector mismatch), so this is maintenance risk rather than a security hole.

Separately, `src/aztec-client.ts` reaches into undocumented SDK internals to make the custom account routable: it indexes `wallet["pxe"]` and `wallet["walletDB"]`, writes a type: “schnorr” row into the wallet database to satisfy the simulation path, and monkey-patches `getAccountFromAddress`. None of these are supported extension points, and none are covered by any test. `setDeclaredSpending` also writes to a single mutable field on a single shared entrypoint instance, so concurrent `createNote` calls in one process race on it — the binding assert makes this fail-closed, degrading to spurious transaction failures rather than misdirected funds, but it is worth knowing before raising throughput.

4.2.11 Build, Artifact, and Test Coverage

```
cd contracts/spending_limit_account
nargo test      # unit tests -- no sandbox, no proving, runs on ARM
nargo compile   # -> target/..-SpendingLimitAccount.json
```

The toolchain must be pinned: `noirup -version 1.0.0-beta.20`, matching both the `noir_version` recorded in the committed artifact and the contract job in CI. The `aztec` dependency tag in `Nargo.toml` must equal the `@aztec/*` npm version in `package.json` — both 4.2.0 today — since a mismatch breaks the contract-class and ABI agreement between the circuit and the SDK. The `schnorr` dependency comes from `noir-lang/schnorr v0.2.0` and is versioned on its own cadence.

The in-contract tests exercise `assert_single_call_matches` against a hand-built `[Field; 31]`: the layout tripwire, the happy path, a declared/actual mismatch, a hidden second call, and an empty payload. Nothing tests `assert_declared_matches_transfer` against a real `AppPayload` (its `function_calls` field is not public outside the `aztec` crate), and nothing at all tests `check_spending_public`, the timelock, or the allowlist — those need a sandbox. On the TypeScript side, `grep -rn "spendingLimit|SPENDING_LIMIT|declared" tests/` returns nothing: the entire on-chain spending-limit path has no test coverage, and `tests/e2e/aztec-client.test.ts` constructs `AztecClient` without a `spendingLimitConfig`. Full-stack verification requires the Aztec sandbox on `x86_64`, since `barretenberg`'s prover crashes under ARM emulation.

Remark about privacy in this mode

The product this contract serves is described as “Aztec *shielded* settlement,” and with a stock account that description is partly earned: the public `finalize` step of `transfer_to_private` carries (`from`, `amount`, `partial_note`), so the sender and the amount are visible on-chain but the recipient exists only inside a note commitment. Recipient privacy is the asymmetry users are buying.

Enabling `SpendingLimitAccount` removes it. The entrypoint enqueues `check_spending_public(declared_amount, declared_recipient)`, and enqueued public calldata travels with the transaction in *plaintext* — the private circuit commits only to a `calldata_hash`, but the sequencer cannot execute a hash, so the raw values ship alongside it and are gossiped network-wide before inclusion. The binding assert then guarantees that `declared_recipient` is exactly the real recipient. An observer therefore learns that the account is a spending-limit account, how often it transfers, when, for how much, and to whom.

This is not a defect to patch but a structural consequence: enforcing an allowlist against `PublicMutable` state requires indexing `recipient_allowlist.at(declared_recipient)`, which requires the recipient in public. One cannot have a public recipient allowlist and a private recipient. Turning on the security feature turns off the privacy, and the repository documents this trade nowhere.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Likelihood		
		Low	Medium	High
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract and circuit development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

5.1 ZK Audit Issue Classification

ZK protocols must adhere to the three principles of zero-knowledge: **Completeness**, **Soundness** and **Zero-Knowledge**. The table below describes the classification of issues in ZK circuits with their corresponding principles of zero-knowledge.

Property	Description	Classification
<i>Completeness</i>	A valid proof must always pass with an honest verifier	Over-constrained
<i>Soundness</i>	An invalid proof should pass only with negligible probability	Under-constrained
<i>Zero-Knowledge</i>	The proof must not reveal any information about the witness	Leakage

6 Issues

6.1 [High] A single set of limits cannot govern different tokens

File(s): `spending_limit_account/src/main.nr`

Description: The contract holds one `max_amount_per_tx`, one `daily_limit` and one `daily_spent`, none of which is keyed by token. The bridge takes the token address as a per-request parameter, so every asset is measured against the same limits and accumulates into the same counter.

Limits and amounts are expressed in base units, and the number of base units per token depends on its decimals. A limit of $1e10$ base units is 10,000 tokens at 6 decimals and $1e-8$ tokens at 18 decimals, so no single value is correct for both.

`daily_spent` also mixes precisions. A transfer of 100 tokens at 6 decimals adds $1e8$, and a transfer of 1 token at 18 decimals adds $1e18$, so the total measures nothing and one high-precision transfer can exhaust the day's allowance for every other asset.

Normalising decimals would not make the counter meaningful either, because the assets differ in value. One unit of a stablecoin and one unit of ether are both counted the same, so a limit expressed in units does not bound the value at risk, which is what a spending limit is intended to do.

Recommendation(s): Key `max_amount_per_tx`, `daily_limit` and `daily_spent` by token address, and bind the token in `assert_declared_matches_transfer` so the counter charged matches the asset moved.

If the deployment is intended for one asset, record the token address at construction and reject requests for any other token.

Status: Fixed

Update from the Client: Commit: `3a6c1fe`. Posted comment is accurate, no amendment needed.

6.2 [High] Daily counter can be reset to zero

File(s): `spending_limit_account/src/main.nr`

Description: When `max_amount_per_tx` is 0 and `daily_limit` is non-zero, a malicious holder of the compromised signing key can set `daily_spent` to any value, including 0, and so transfer more than `daily_limit` permits. Both conditions are required, because the daily branch is skipped entirely when `daily_limit` is 0.

Two root causes combine:

- The `declared_amount` is not range-checked when the per-transaction cap is disabled. The check appears only inside the per-transaction branch. The daily branch validates the sum rather than the addend;

```

1  if max_per_tx != 0 { assert_amount_within_limit(declared_amount, max_per_tx); }
2  ...
3  spent = spent + declared_amount;
4  assert_amount_within_limit(spent, daily_limit);
5  self.storage.daily_spent.write(spent);

```

`spent` is a `Field`, so the addition is modulo the BN254 prime p . An addend of $p - \text{daily_spent}$ produces a sum of 0, which satisfies `assert_amount_within_limit` because 0 is in range.

- The `assert_single_call_matches` does not constrain the target. It reads the args hash and tests whether `target_address` is non-zero. It does not read `function_selector`, and it does not compare `target_address` against any permitted value, so the executed call need not be a token transfer. A contract exposing an empty function that accepts two `Field` arguments satisfies the guard;

Exploitation:

1. Deploy a contract exposing `fn noop(a: Field, b: Field)` with an empty body;
2. Read `daily_spent` from public state;
3. Submit a transaction whose payload contains one call to `noop(recipient, p - daily_spent)`, where `recipient` is an allowed address. The guard is satisfied, the callee performs no transfer, and `check_spending_public` writes `daily_spent = 0`;
4. Submit an ordinary transfer of up to `daily_limit`;
5. Repeat from step 2;

Step 3 moves no value and reads no balance. The declared amount is passed as a call argument only.

Recommendation(s): Consider addressing the issue by resolving the root causes:

- Range-check `declared_amount` before either branch;

- Constrain the callee in `assert_single_call_matches` by comparing `serialized[base + 1]` against the selector for `transfer_to_private`, which is a compile-time constant and adds no runtime cost. Additionally, consider adding a whitelist of permitted target addresses, so the payload cannot call an arbitrary contract;

Status: Fixed

Update from the Client: Commits: 8f7105b (root cause 1) and 490c88c (root cause 2)

AMEND. The posted comment says `declared_amount` is range-checked "at the top of `check_spending_public`". It is not. The check is the `u128` parameter type, one hop earlier on the private entrypoint. A check at the top of the public function would be a no-op.

Amendment:

Both root causes are closed. The per-tx ceiling is unconditional: zero is not a storable limit, so the `if max_per_tx != 0` branch that let `declared_amount` reach the daily arithmetic unchecked cannot exist. The counter is `u128`, so accumulation is a checked add that traps on overflow rather than wrapping mod `p`, which removes the `p - daily_spent` addend. Separately the payload guard now reads `serialized[base + 1]` and pins the selector to `transfer_to_private` as a compile-time constant, plus `is_public`, `hide_msg_sender` and `is_static`. Pinning `is_public` is not defence in depth: `execute_calls` omits the selector on the public branch, so leaving it unpinned would make the selector pin dead data there. The `noop(a: Field, b: Field)` exploit no longer satisfies the guard.

Update from Nethermind: We accept the change. The security review was performed from the perspective of the stolen account's signing key, since this is the main focus of the introduced functionality. The contract itself states: "if the bridge's signing key is compromised, limit the damage." The admin in that scenario remains safe as a trusted third-party address. The malicious reset by admin would be exploitable only if both the admin account and the limiting account's private key are compromised.

6.3 [High] u64 limits are too small for 18-decimal tokens

File(s): `spending_limit_account/src/main.nr`

Description: The contract stores spending limits as `Field` values but validates them as `u64`, while token balances and transfer amounts use `u128`. For a token with 18 decimals, the `u64` check restricts every limit to 18.45 tokens.

The check is implemented in `assert_amount_within_limit`, which casts the amount and the limit to `u64` and asserts that neither loses precision:

```

1 let a = amount as u64;
2 assert(a as Field == amount, "Amount exceeds u64 range");
3 let l = limit as u64;
4 assert(l as Field == limit, "Limit exceeds u64 range");
5 assert(a <= l, "Amount exceeds spending limit");

```

The same check is applied by `initialize_public_state` to the limits supplied at deployment.

The maximum value a `u64` can hold is 1.8×10^{19} base units, which is 18.45 tokens at 18 decimals. Limits above this value cannot be configured, because deployment fails with `Daily limit exceeds u64 range`. Individual transfers and cumulative daily totals above the same figure are rejected.

The number of decimals is set per token, so the effect varies by asset. At 6 decimals the ceiling is 1.8×10^{13} tokens. At 18 decimals, as used by WETH and DAI, no usable limit can be set.

Recommendation(s): Store `max_amount_per_tx` and `daily_limit` as `u128` and perform the range checks at that width, matching the type used by the Token contract.

If `u64` is required, document the maximum number of decimals supported and reject deployment for tokens that exceed it.

Status: Fixed

Update from the Client: Commit: 8f7105b

AMEND. The posted comment says `daily_spent` moves to `PublicMutable<u128>` and `assert_amount_within_limit` range-checks at `u128`. Neither survives. `daily_spent` no longer exists as a scalar (it is the bucket array from the M1 fix), and `assert_amount_within_limit` was deleted.

Amendment:

Limits move from `PublicMutable<Field>` to `PublicMutable<u128>`, matching the amount type the Token contract uses. The type is now the range, so `assert_amount_within_limit` is gone and every check is a plain comparison. `declared_amount` becomes a `u128` parameter on both entrypoint and `check_spending_public`. As a parameter the framework constrains it below 2^{128} before any contract line runs, where a `Field` as `u128` cast would silently truncate. Zero is rejected in both limit slots, which removes the precondition for the counter-reset finding.

6.4 [Medium] Fixed Epoch Daily Limit Allows Up to 2× Cap Bypass

File(s): `spending_limit_account/src/main.nr`

Description: The account contract's daily volume cap is documented as a rolling 24-hour window. CLAUDE.md describes PXE-BRIDGE_DAILY_LIMIT as a "24h rolling volume cap; circuit-breaker pauses bridge", and explicitly states that the on-chain limits "enforce independently" the same guarantee as the application-level.

The function `check_spending_public` does not follow the same model. Instead of a sliding window, it resets the entire counter whenever the current block crosses a fixed epoch boundary computed from the absolute block number:

```

1  global BLOCKS_PER_EPOCH: u64 = 7200; // ~24h at 12s/block
2
3
4  #[external("public")]
5  #[noinitcheck]
6  fn check_spending_public(//ok
7      declared_amount: Field,
8      declared_recipient: AztecAddress,
9  ) {
10
11      // ...
12      let daily_limit = self.storage.daily_limit.read();
13      if daily_limit != 0 {
14          let current_block = self.context.block_number() as u64;
15          let epoch = current_block / BLOCKS_PER_EPOCH;
16          let stored_epoch = self.storage.current_epoch.read() as u64;
17
18          let mut spent = if epoch != stored_epoch {
19              self.storage.current_epoch.write(epoch as Field);
20              0 as Field
21          } else {
22              self.storage.daily_spent.read()
23          };
24
25          spent = spent + declared_amount;
26          assert_amount_within_limit(spent, daily_limit);
27          self.storage.daily_spent.write(spent);
28      }

```

`epoch = current_block / BLOCKS_PER_EPOCH` divides the chain's history into fixed 7200-block windows aligned to block 0, not to the timestamp of the account's own spending activity. The counter resets to zero in full at each boundary, regardless of how recently the previous spend occurred.

Concrete Example: assume `daily_limit = 1000` and an epoch boundary at block 14400 (epoch 1 spans blocks 7200–14399, epoch 2 spans 14400–21599).

1. **Block 14399** (last block of epoch 1): a transaction with `declared_amount = 1000` is sent. `epoch = 14399 / 7200 = 1`, matching `stored_epoch` (assume this is the first spend recorded in epoch 1, so `daily_spent=0`). `spent = 0 + 1000 = 1000`, which passes (`1000 <= 1000`). `daily_spent` is written as 1000;
2. **Block 14400** (the very next block, 12 seconds later): another transaction with;

`declared_amount = 1000` is sent. `epoch = 14400 / 7200 = 2`, which differs from `stored_epoch = 1`, so `spent` is reset to 0 before adding. `spent = 0 + 1000 = 1000`, which again passes.

Total moved: **2000 units across two consecutive blocks (12 seconds)**, double the configured `daily_limit`, simply by timing two transactions around a publicly-known epoch boundary (the current block number is public information).

This directly *undermines the purpose of the daily cap as described in the **threat model*** at the top of `main.nr` (limiting damage from a compromised signing key): an attacker who has the key can extract up to 2x `daily_limit` in a matter of seconds by simply waiting for (or computing) the next epoch boundary. <!--, rather than being throttled to `daily_limit` per rolling day as intended.-->

Recommendation(s): Implement a rolling window in the contract, e.g., track the block/timestamp at which the current window started and only reset when `current_block - window_start >= BLOCKS_PER_EPOCH`, rather than resetting on every `block_number / BLOCKS_PER_EPOCH` boundary crossing. Or document the on-chain limit explicitly as a fixed calendar epoch (not "rolling"), and size `daily_limit / BLOCKS_PER_EPOCH` with the possibility of a 2x burst at each boundary factored into the threat model.

Status: Fixed

Update from the Client: Commit: `bfc41ac`

AMEND. The posted comment describes `window_start` anchored to the account's first spend, and "24 sub-buckets bounding the burst to $L \times 25/24$ ". Neither shipped, and the 24-bucket figure is arithmetically false.

Replacement:

The fixed epoch is replaced by a bucketed sliding window of 25 buckets of 3600 seconds, anchored on `context.timestamp()` rather than block height, because `BLOCKS_PER_EPOCH` assumed exactly 12s blocks and Aztec's `block_number()` documentation says intervals must not be relied upon. Correction to our earlier figure: twenty-four hourly buckets do not bound the daily total. The minimum lookback of an N -bucket window of width W is

$$(N - 1)W,$$

which for $N = 24$ is 23 hours, so two full-limit spends separated by 82801 seconds both pass and a rolling 24-hour interval still admits $2L$. The invariant that gives an exact bound is

$$(N - 1)W \geq 86400,$$

which with hourly buckets requires 25. At 25 the minimum separation is 86401 seconds, so $2L$ never fits inside 24 hours. The $25/24$ figure is real, but it is the conservatism of the design (maximum lookback runs to 25 hours), not a burst allowance. `window_parameterisation_is_exact` asserts the invariant so a gas-motivated retune to 24 cannot compile.

6.5 [Medium] Recipient is published in public calldata

File(s): `spending_limit_account/src/main.nr`

Description: The entrypoint enqueues `check_spending_public(declared_amount, declared_recipient)`. Arguments to an enqueued public call are visible on chain, so every transfer publishes the recipient's address. For a token transfer, `assert_declared_matches_transfer` binds that value to the recipient argument of the executed call, so the published address is the one that receives the funds. The cause is the allowlist lookup rather than the public call itself. `recipient_allowlist.at(declared_recipient).read()` reads `PublicMutable` state, and deriving a Map slot requires the plaintext key, so the recipient has to be passed in the clear. The same property makes the allowlist readable: anyone can compute the slot for a guessed address and check whether it is permitted. In effect the recipient is exposed publicly.

Recommendation(s): If recipient privacy is intended, move the allowlist check into the private entrypoint. Consider following changes:

- Create the `recipient_allowlist` with type `DelayedPublicMutable<bool>`. Writes take effect after a configured delay, which makes a private read provably still current at inclusion time. See [Shared State](#);
- Do the lookup during private execution in entrypoint and shorten the public call to `check_spending_public(declared_amount)`. Keep `declared_recipient` in the signed combined hash so the binding survives;
- Leave the other slots public, since `max_amount_per_tx` and `daily_limit` are global rather than per-recipient, and `daily_spent` and `current_epoch` need an atomic read-modify-write that private execution cannot provide;

The cost is revocation lag, since removing a recipient would only take effect after the delay.

Alternatively, use a Merkle proof to privately prove inclusion against the public root.

Status: Fixed

Update from the Client: Fixed in commit `b0a24bc`.

6.6 [Low] cancellable Transactions Cannot Be Cancelled

File(s): `spending_limit_account/src/main.nr`

Description: The account contract exposes a cancellable flag and emits a transaction cancellation nullifier derived from `app_payload.tx_nonce`:

```
1 let tx_nullifier = poseidon2_hash_with_separator(
2     [app_payload.tx_nonce],
3     DOM_SEP__TX_NULLIFIER,
4 );
5 self.context.push_nullifier(tx_nullifier);
```

This suggests support for transaction cancellation through nullifier conflicts. However, the contract does not provide any execution path that allows the account owner to cancel the transaction. The only entrypoint capable of producing the nullifier is `entrypoint()`, which requires a valid transfer payload and enforces:

* exactly one non-empty call; * a transfer whose arguments match `declared_amount` and `declared_recipient`;

As a result, cancelling a pending transaction is only possible by executing another constrained transfer that reuses the same `tx_nonce`, it always requires spending real funds to a recipient already present in `recipient_allowlist`, within `max_amount_per_tx` / `daily_limit`. Since that replacement transfer is itself gated by the same spending-limit checks the cancellation mechanism should be able to bypass in an emergency, there exist reachable states (e.g. `daily_limit` exhausted, or no allowlisted recipient available) in which cancellation becomes provably impossible. Any attempt to submit a truly empty payload (the standard no-op cancellation pattern) is rejected by `assert_declared_matches_transfer()`.

Recommendation(s): Branch on the number of non-empty calls in `app_payload` before enforcing `assert_declared_matches_transfer /execute_calls /check_spending_public`: if the payload contains zero non-empty calls, skip those checks entirely (requiring only `cancellable == true`) and proceed directly to the nullifier push. <!--Signature verification is unaffected, since it

already runs unconditionally before this branch and covers `app_payload.hash()` regardless of its contents. This reuses the existing entrypoint signature and authentication path rather than introducing a second one.-->

Status: Fixed

Update from the Client: Not yet implemented. Deleted the unreachable cancellable branch in commit 7f057ec.

6.7 [Low] Documented timelock does not cover the allowlist

File(s): `spending_limit_account/src/main.nr`

Description: `TODO.md` marks this as done:

> **Timelock for parameter changes** – limit/ **allowlist** updates take effect after N blocks, giving operators time to detect unauthorized changes."

TIMELock_BLOCKS gates only `propose_limits` and `apply_limits`. `set_recipient` writes straight through, so allowlist updates take effect after zero blocks.

Recommendation(s): Either implement the documented behaviour or correct the document.

Status: Fixed

Update from the Client: Commit: 1bd4a3c

AMEND. The posted comment hedges with "removals take effect immediately (sometimes delayed* per M2)". Drop the hedge.

Amendment:

Delivered as originally stated, with no caveat. Additions go through `propose/apply` with the timelock. Removals are immediate and additionally invalidate in-flight transactions whose hint carried the removed address, because the binding is re-evaluated against current state at inclusion time. Additions do not invalidate in-flight transactions: a slot empty at proof time reads zero in the hint and passes whatever was written later. The asymmetry is deliberate. The threat is a compromised admin adding an address they control, and a compromised admin must not be able to delay a legitimate revocation.

Evidence: invalidates a transfer proven before the recipient was removed (de235a8, tightened in a41fdc5). Two transfers are proven together, against the same allowlist and anchor state, and submitted either side of the removal. The first lands. The second settles as reverted, accepted with a valid proof, included in a block, killed by the public phase, and the balance is unchanged. The node reports a bare reverted and populates no error, so the reason comes from the pair rather than the receipt: identical transactions, identical proving conditions, one variable.

6.8 [Info] The single-call restriction in entrypoint may interfere with deployment

File(s): `spending_limit_account/src/main.nr`

Description: `assert_single_call_matches` requires an exact count rather than an upper bound:

```
1 assert(nonempty_calls == 1, "Expected exactly one call in payload");
2 assert(matching_calls == 1, "Transfer does not match declared spending");
```

So entrypoint can carry one declared transfer and nothing else.

An account that pays for its own deployment is called through its own entrypoint, because only the account can name itself fee payer and the multicall contract that assembles the deployment cannot. That call reaches `assert_declared_matches_transfer` after the signature check and the fee handling have both succeeded, and what it carries is fee-related rather than a transfer:

- Fee Juice claimed in the same transaction: The payload holds one call to the Fee Juice contract, taking four arguments;
- A fee-paying contract: The payload holds one call to that contract, taking none;
- Balance funded in advance: No fee call is needed, but the entrypoint is still called to designate the fee payer, so the payload holds no calls at all. `AppPayload` is a fixed `[FunctionCall; 5]`, and unused slots are padded with `target_address == 0`, which is the predicate the guard counts;

The first two cases fail the second assertion, because neither a four-argument nor a zero-argument call can reproduce a two-field `hash_args([declared_recipient, declared_amount])`. The third fails the first assertion, with zero non-empty calls. The declared values are not the cause: no value of the pair makes a fee call resemble a transfer.

The bridge currently deploys this way. The comment preceding the deployment call in `../src/aztec-client.ts` records the intent: **"triggers the multicall self-deploy path where the contract is constructed before it pays for its own fee"**. Whether that remains the deployment strategy is outside the scope of this review; the point recorded here is that the restriction constrains it.

The issue is marked as informational, since the problem seems to be in the out-of-scope part, which may be not final.

Recommendation(s): Consider deploying from a separate funded account, which avoids the restriction, because only the constructor is called and the account address is unaffected.

Status: Fixed

Update from the Client: Commit: 1bc93ba

AMEND. Two errors in the current draft. "The address is unaffected" is false: the constructor gained `token_address`, `seed_recipient` and `initial_min_anonymity`, so the account address moves and existing `FEE_JUICE_CLAIM` values are invalid. The "two calls" diagnosis is also wrong. On the v5.1.0 self-deploy path the payload carries one call, so `nonempty_calls == 1` passed and the revert was on the next assertion.

Replacement:

The account is deployed by a separately funded Schnorr deployer, which runs only the constructor, so the guard is never reached. `DeployAccountMethod` hardcodes `universalDeploy`, so which account pays does not move the address. The restriction reaches further than deployment. A fee payment method contributes its own call, and `BaseWallet` merges that payload into the same `AppPayload` the entrypoint receives (`mergeExecutionPayloads`). `SponsoredFPC`'s `sponsor_unconditionally` and `FeeJuicePaymentMethodWithClaim`'s `claim_and_end_setup` therefore both arrive beside the transfer and trip `nonempty_calls == 1`. The account can only transact on `PREEXISTING_FEE_JUICE`, and it cannot obtain that balance itself, because claiming is one of the calls the guard forbids. This is an operational constraint, not a test artifact: whoever bridges fee juice must also claim it on the account's behalf. `FeeJuice.claim` names its recipient in an argument rather than taking the caller, so a third party can do this. The e2e suite funds the account exactly this way. `PREEXISTING_FEE_JUICE` is also the only entrypoint branch that calls `end_setup()`, so this is the path the phase-ordering test exercises.

7 Documentation Evaluation

Software documentation refers to the written or visual information describing software's functionality, architecture, design, and implementation. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Protocols can use various types of software documentation for their circuits. Some of the most common types include:

Circuit Specifications: A documentation that outlines the purpose, functionality, and constraints of the circuit:

- i) High-level description - What the circuit is designed to achieve, e.g., proving membership in a Merkle tree, verifying a signature, etc. Description about inputs and outputs (public and private signals);
- ii) Mathematical model - Description of the mathematical computations and proofs being implemented. In addition, describe the cryptographic primitives that are used, e.g., Poseidon hash, EdDSA verification.
- iii) Expected behavior - Present a detailed explanation of how the circuit behaves under normal, edge, and adversarial conditions.

Documentation of the Input and Output: The documentation should describe details of the inputs and outputs of the circuit.

- i) Public inputs: List of public inputs and their expected values and ranges. Also, clarify constraints on the public inputs (e.g., nonzero).
- ii) Private inputs: List of private input signals and their roles in the circuit. The documentation should also include validation requirements for private inputs within the circuit.
- iii) Outputs: A description of the outputs, their intended use and their relationship with the inputs.

Performance Metrics: Describe details on the prover and verifier performance such as, time taken to generate proofs, memory usage during proof generation, time taken to verify proofs, etc.

Testing Documentation: Testing documentation is a document that provides information about how the circuits were tested. It includes details on the test cases that were used, coverage metrics showing which parts of the circuit have been tested, and any issues that were identified during testing.

Technical whitepaper: A technical whitepaper is a comprehensive document that describes the design and technical details of the protocol. It includes information on the purpose of the protocol, contracts, its architecture, its components, and how they interact with each other;

Summary: These types of documentation are essential for circuit development and maintenance. They help ensure that the circuit is properly designed, implemented, and tested. The documentation provides a reference for developers who need to modify the circuit in the future.

Remarks about the PXE-Bridge documentation

The **PXE-Bridge** protocol documentation was made available through [whitepaper](#) and MD files with a high-level overview. In addition, the team provided audit-specific documentation, detailing the core contracts and files, outlining the flow step by step with comments in the code.

8 Test Suite Evaluation

8.1 Tests Output

```
> @xochi-fi/pxe-bridge@0.1.1 test
> vitest run
```

```
RUN v4.1.4 /Users/luciana/audits/nm-1019-axol-pxe-bridge/pxe-bridge
```

```
Test Files  8 passed (8)
Tests       134 passed (134)
Start at    19:45:25
Duration    391ms (transform 653ms, setup 0ms, import 929ms, tests 194ms, environment 0ms)
```

```
Noir contract tests (contracts/spending_limit_account) - nargo test
```

```
=====
[spending_limit_account_contract] Running 5 test functions
[spending_limit_account_contract] Testing SpendingLimitAccount::function_call_serialize_layout ... [0m[32mok[0m
[spending_limit_account_contract] Testing SpendingLimitAccount::mismatched_transfer_reverts ... [0m[32mok[0m
[spending_limit_account_contract] Testing SpendingLimitAccount::hidden_second_call_reverts ... [0m[32mok[0m
[spending_limit_account_contract] Testing SpendingLimitAccount::empty_payload_reverts ... [0m[32mok[0m
[spending_limit_account_contract] Testing SpendingLimitAccount::single_matching_call_passes ... [0m[32mok[0m
[spending_limit_account_contract] [0m[32m5 tests passed[0m
```

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our Cryptography Research team is dedicated to continuous internal research while fostering close collaboration with external partners. The team has expertise across a wide range of domains, including cryptography protocols, consensus design, decentralized identity, verifiable credentials, Sybil resistance, oracles, and credentials, distributed validator technology (DVT), and Zero-knowledge proofs. This diverse skill set, combined with strong collaboration between our engineering teams, enables us to deliver cutting-edge solutions to our partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

Learn more about us at nethermind.io.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.